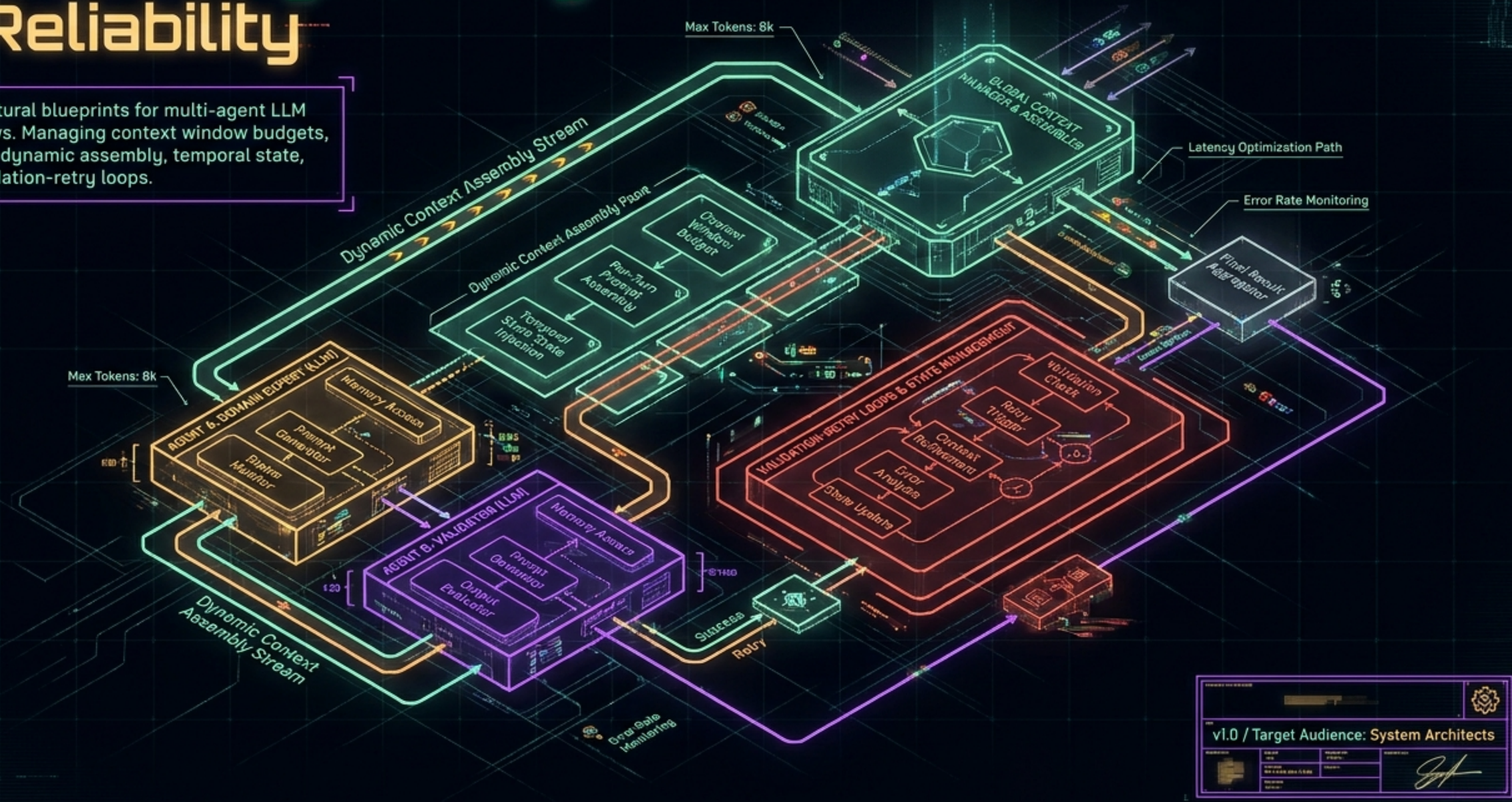
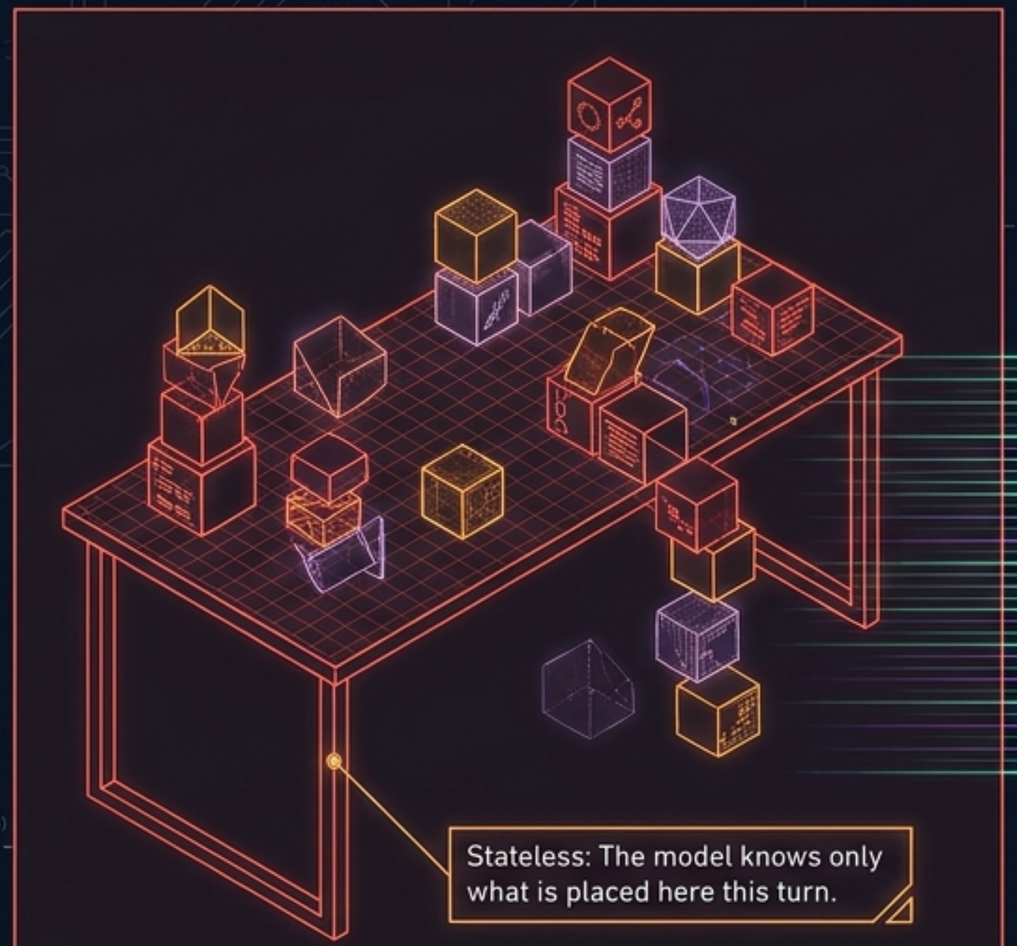
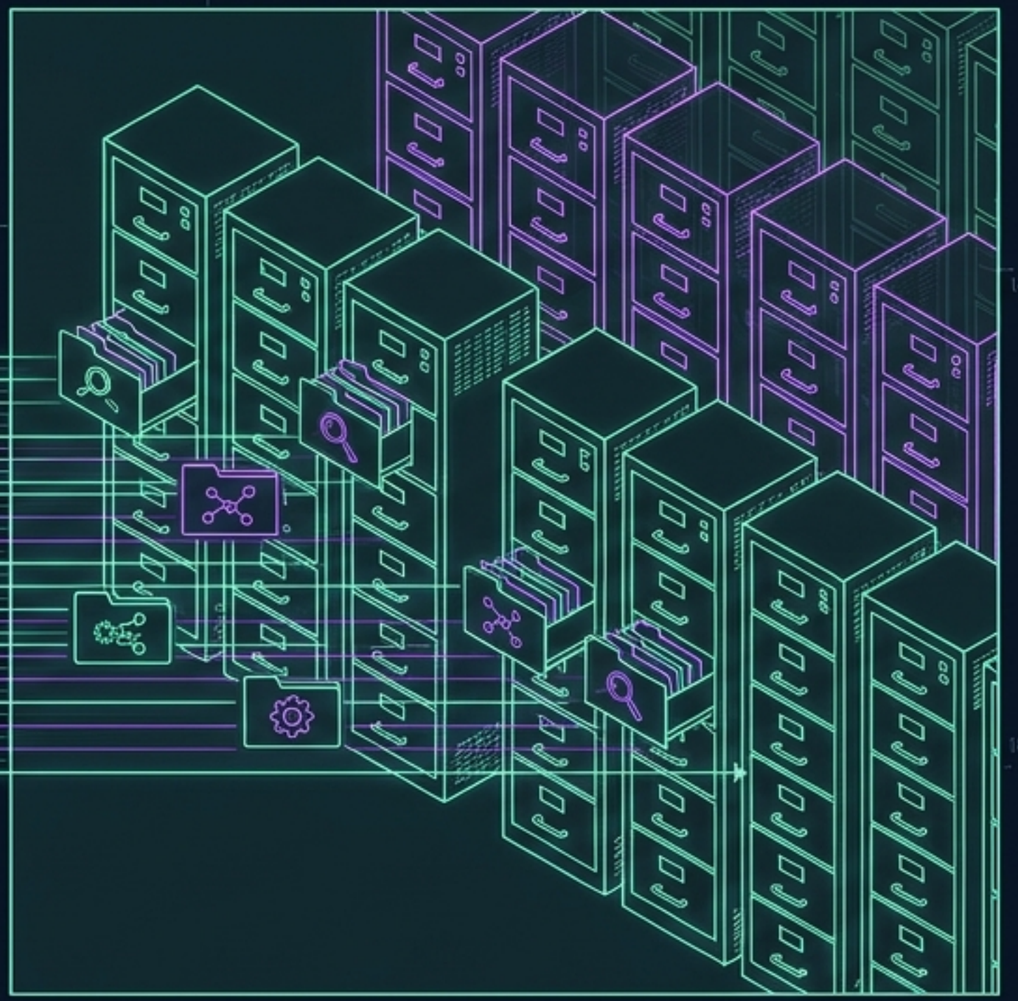
Context Engineering & Reliability

Architectural blueprints for multi-agent LLM workflows. Managing context window budgets, per-turn dynamic assembly, temporal state, and validation-retry loops.







Context Engineering = Deciding what goes on the desk, in which pile, in what order, and what gets filed away with a retrieval label.

Prompt Engineering (Authoring)	Wording/examples written once, reused.
Context Engineering (Per-turn assembly)	Selects, orders, compresses, and isolates window contents on every call.
State (Exact working data)	Key/value data read deterministically (IDs, flags). Never approximated.
Memory (Long-term recall)	Extracted facts across sessions. Probabilistic recall.
Retrieval/RAG (Shared knowledge)	Documents valid for all users.
Reliability Engineering (Failure handling)	Structured errors, validation, escalation.



The Window Limit: Opus 4.7 tokenizer consumes roughly 30% more tokens for the same text than legacy models.



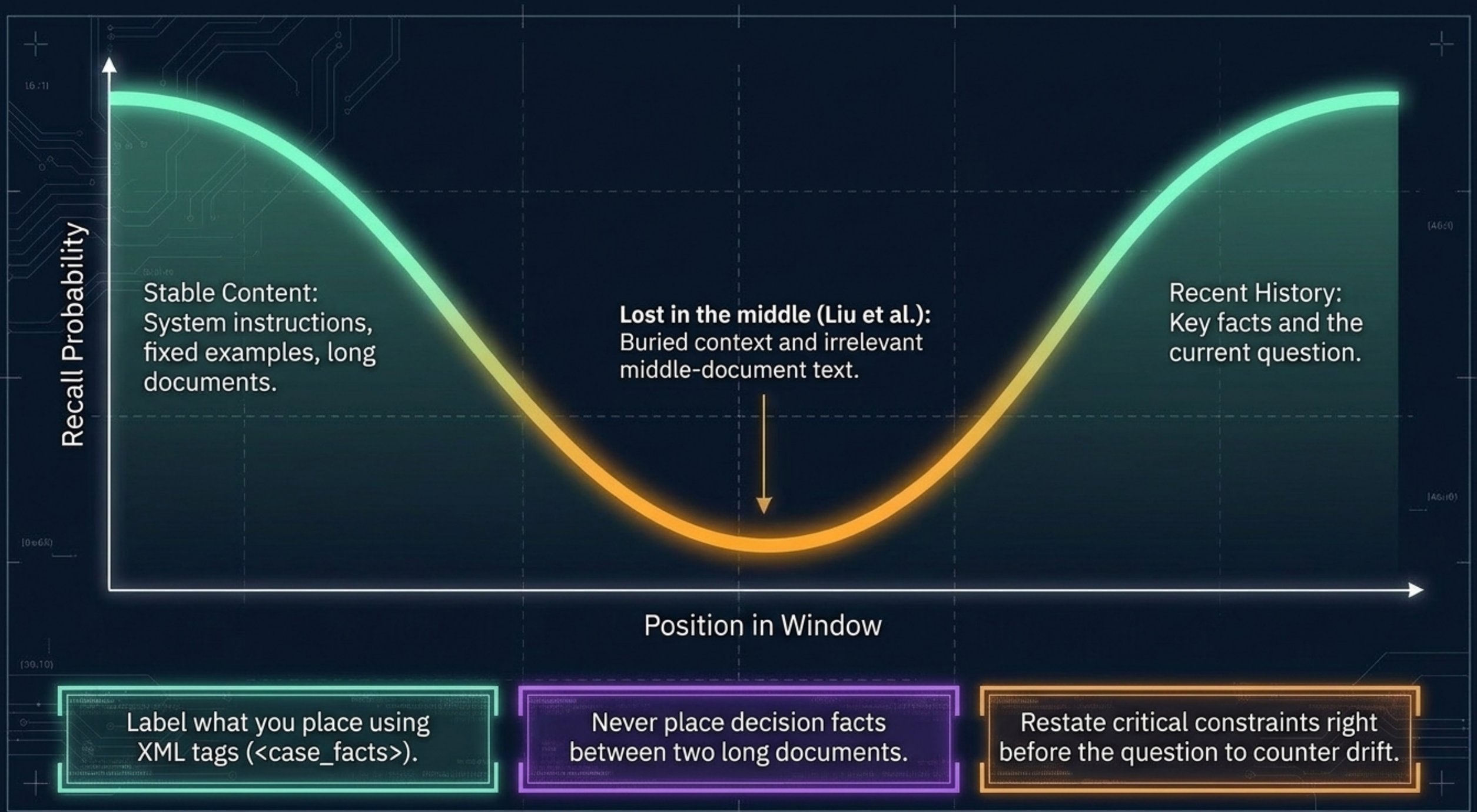
Fixed Tax: 30 verbose tool definitions consume budget before the first user word.

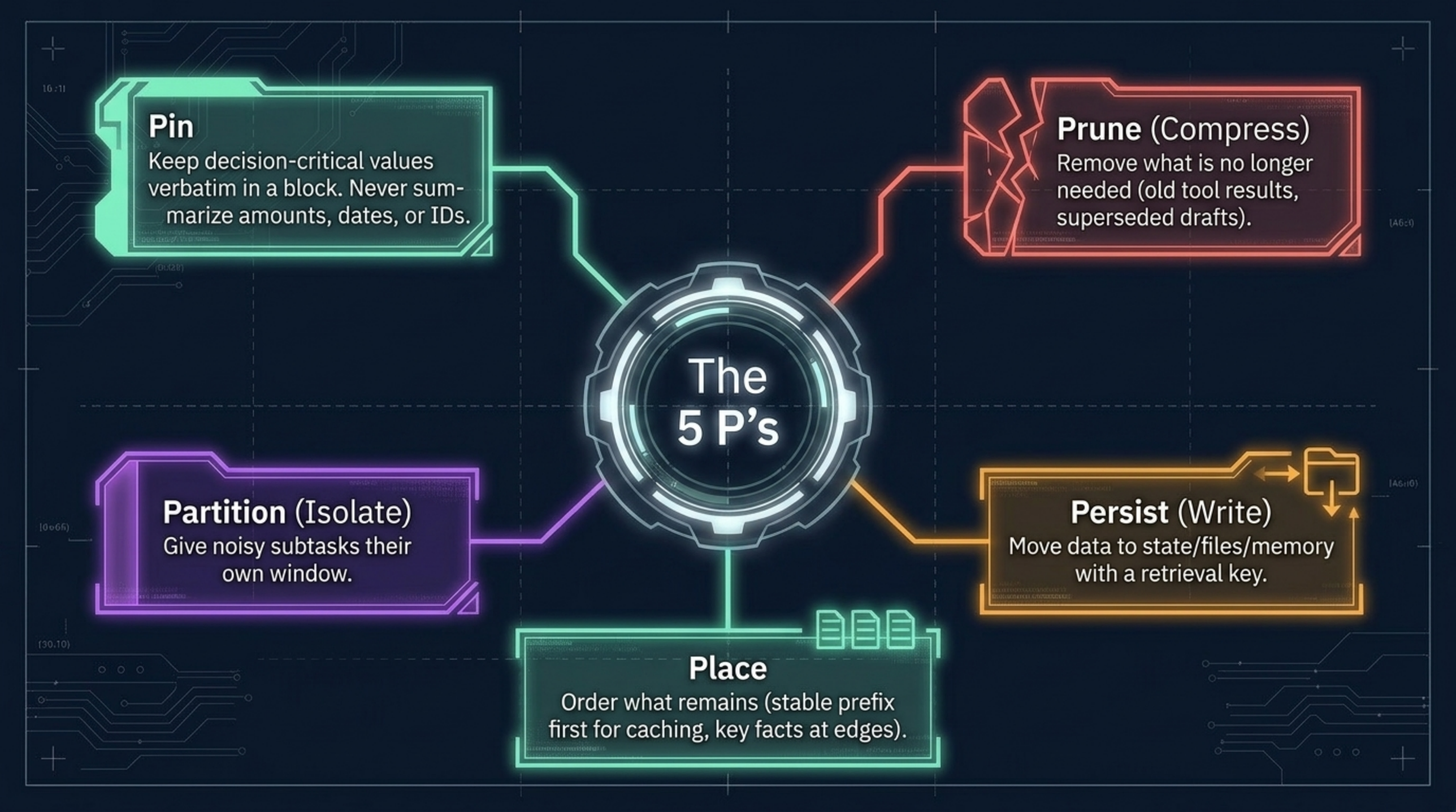


Reserved allocation for max_tokens / max_output_tokens.

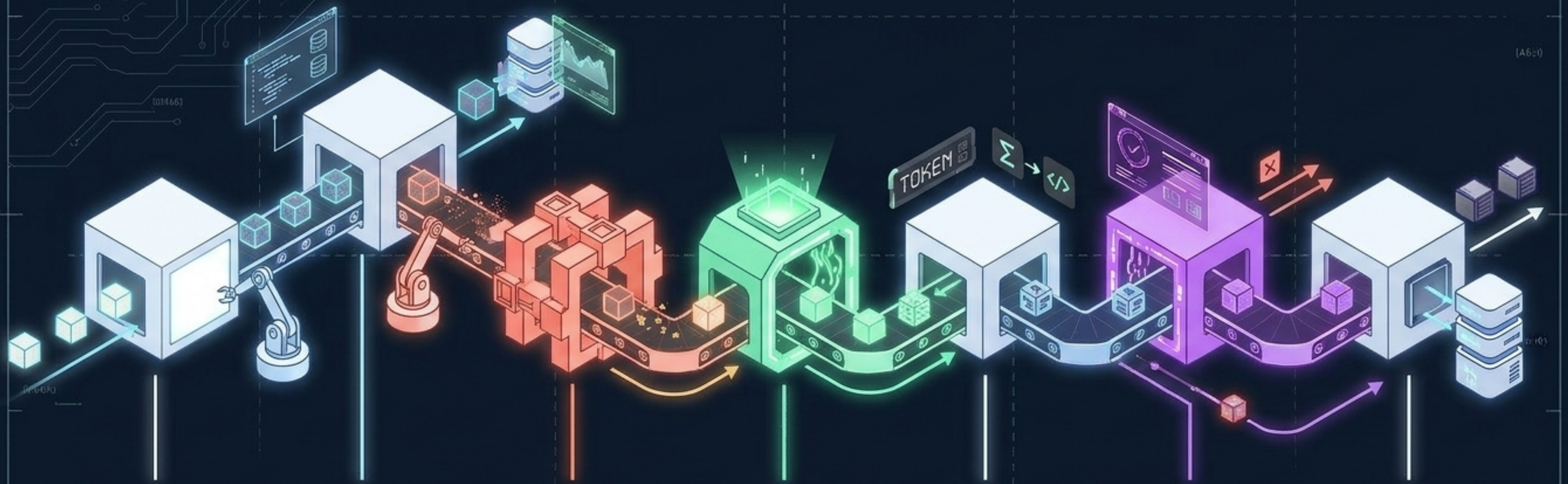


Recall degrades well before the hard limit. Irrelevant tokens lower recall of relevant ones.


Recall Probability
Stable Content:
System instructions, fixed examples, long documents.
Lost in the middle (Liu et al.):
Buried context and irrelevant middle-document text.
Recent History:
Key facts and the current question.
Position in Window
Label what you place using XML tags (<case_facts>).
Never place decision facts between two long documents.
Restate critical constraints right before the question to counter drift.


The 5 P's
Pin
Keep decision-critical values verbatim in a block. Never summarize amounts, dates, or IDs.
Prune (Compress)
Remove what is no longer needed (old tool results, superseded drafts).
Partition (Isolate)
Give noisy subtasks their own window.
Persist (Write)
Move data to state/files/memory with a retrieval key.
Place
Order what remains (stable prefix first for caching, key facts at edges).

Data Processing Pipeline



1. Load:
Fetch session events, exact state, pinned facts block.

2. Retrieve:
Inject relevant memories/RAG.

3. Prune:
Clear stale tool results; compact old turns.

4. Place:
Order blocks (stable prefix -> pinned -> retrieved -> history -> input).

5. Call:
Token count check; execute.

6. Validate:
Check schemas and business rules. Route errors.

7. Persist:
Commit state changes, update scratchpads, extract memories.

The Failure

Engineer states 4.2 MW load.

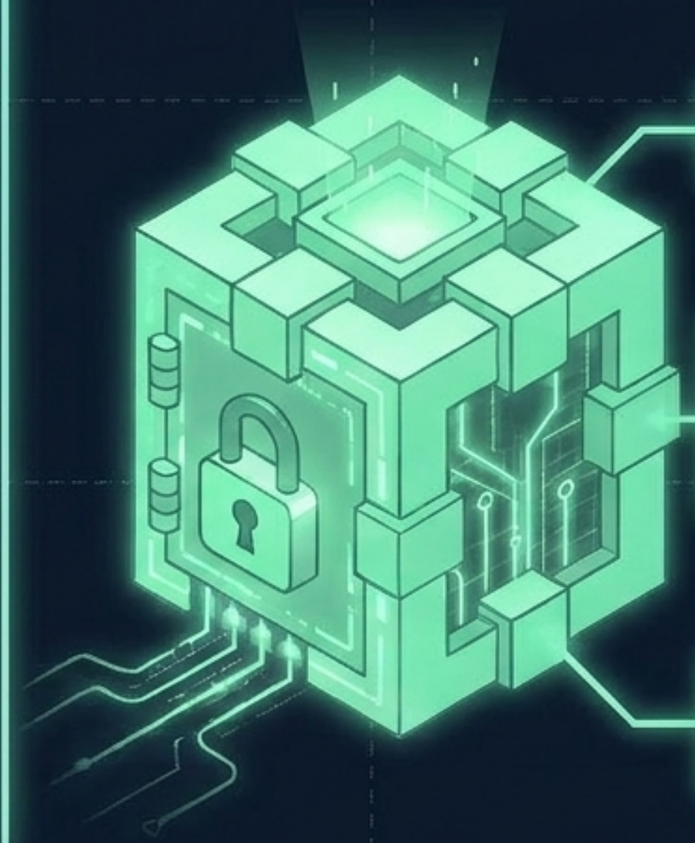
↓

Summary compresses it.

↓

Agent proposes switching plan for “roughly 4 MW” and drops the vital dialysis clinic.

The Fix

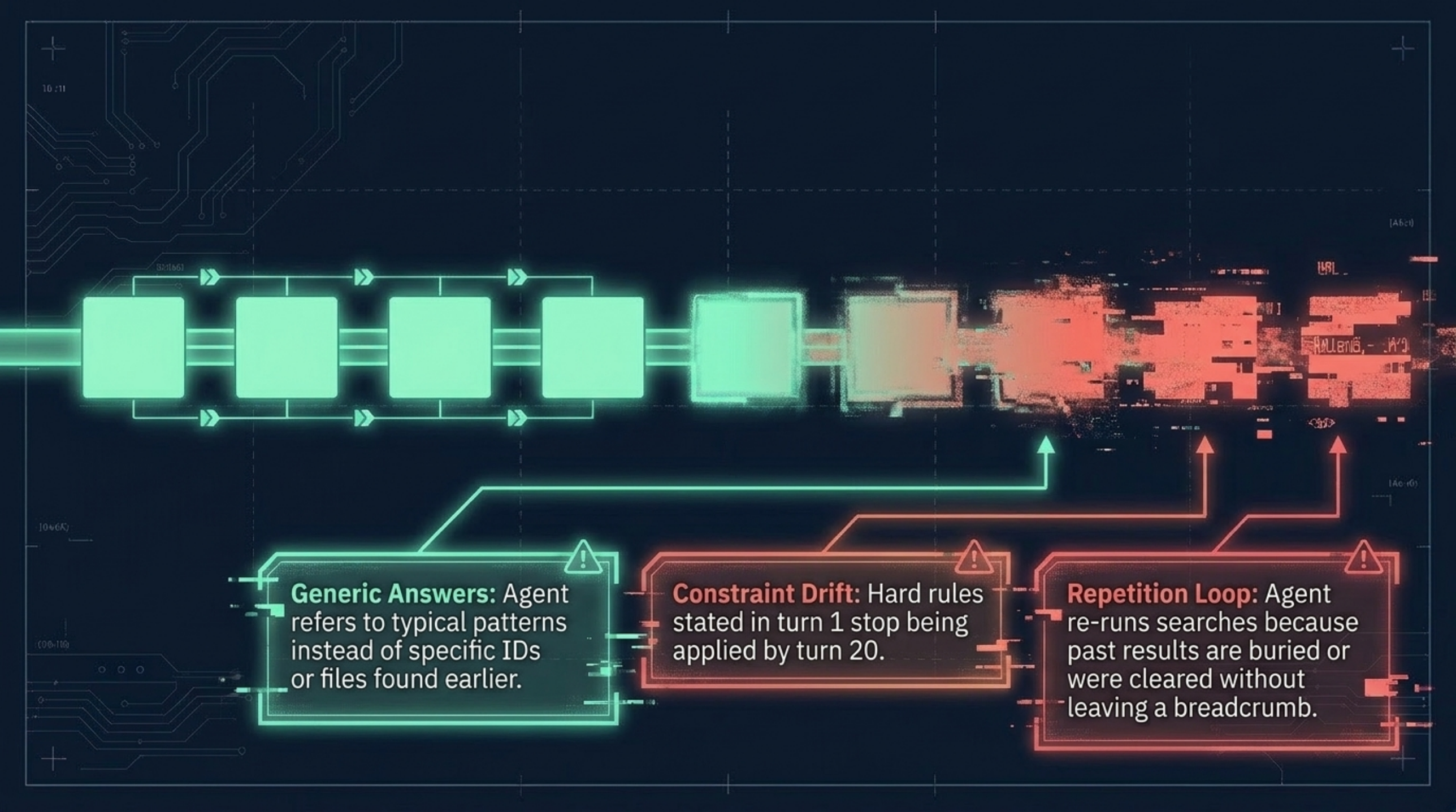


The Pinned Facts Block

Owned by the harness. Re-injected at the top of every call.

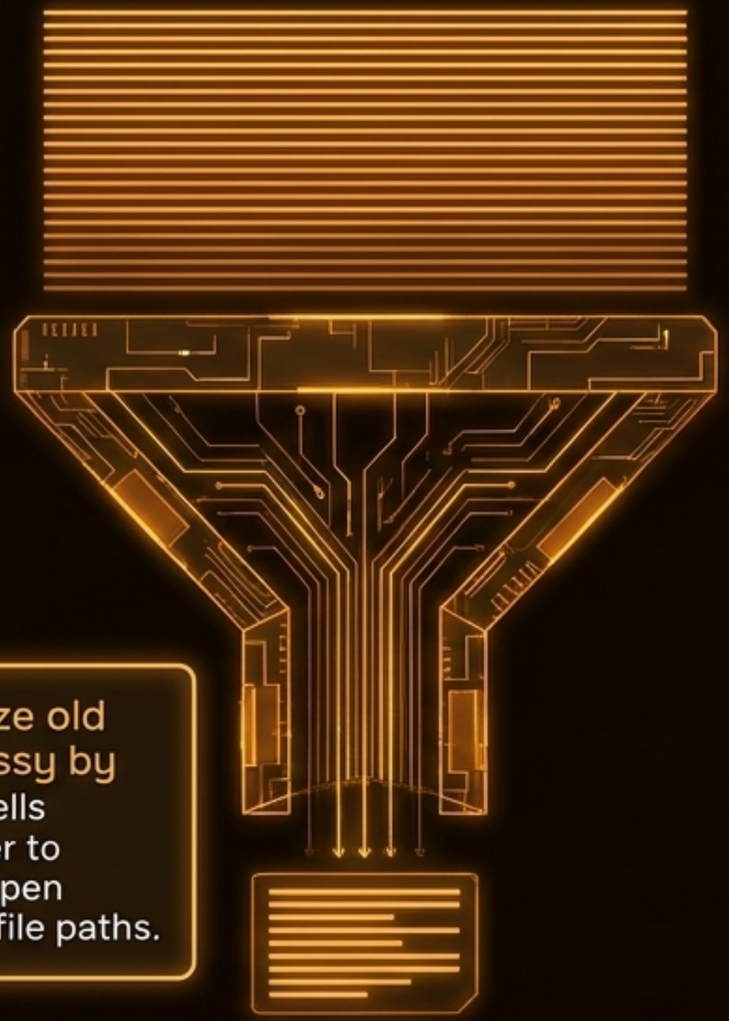
Never touched by summarization.

⚠ Provenance inside the block: Record who stated each value and when. **Trusting summaries for exact amounts is a fatal anti-pattern.**


Generic Answers: Agent refers to typical patterns instead of specific IDs or files found earlier.
Constraint Drift: Hard rules stated in turn 1 stop being applied by turn 20.
Repetition Loop: Agent re-runs searches because past results are buried or were cleared without leaving a breadcrumb.

Compaction

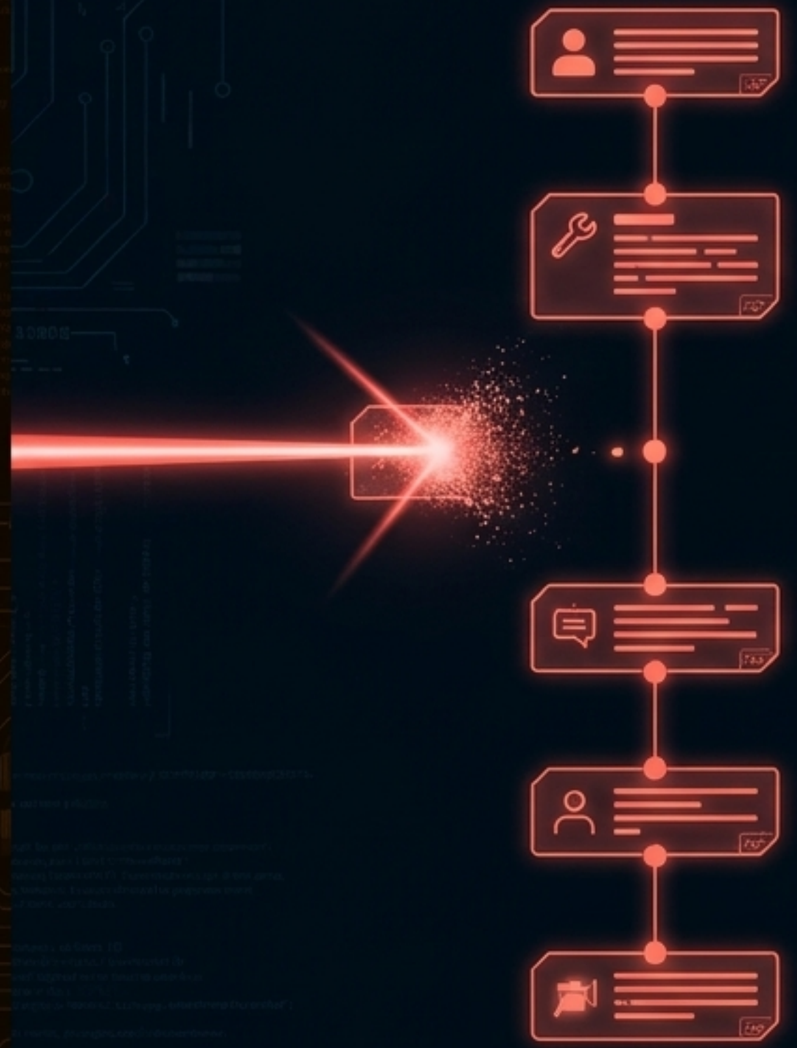
Raw Conversation History



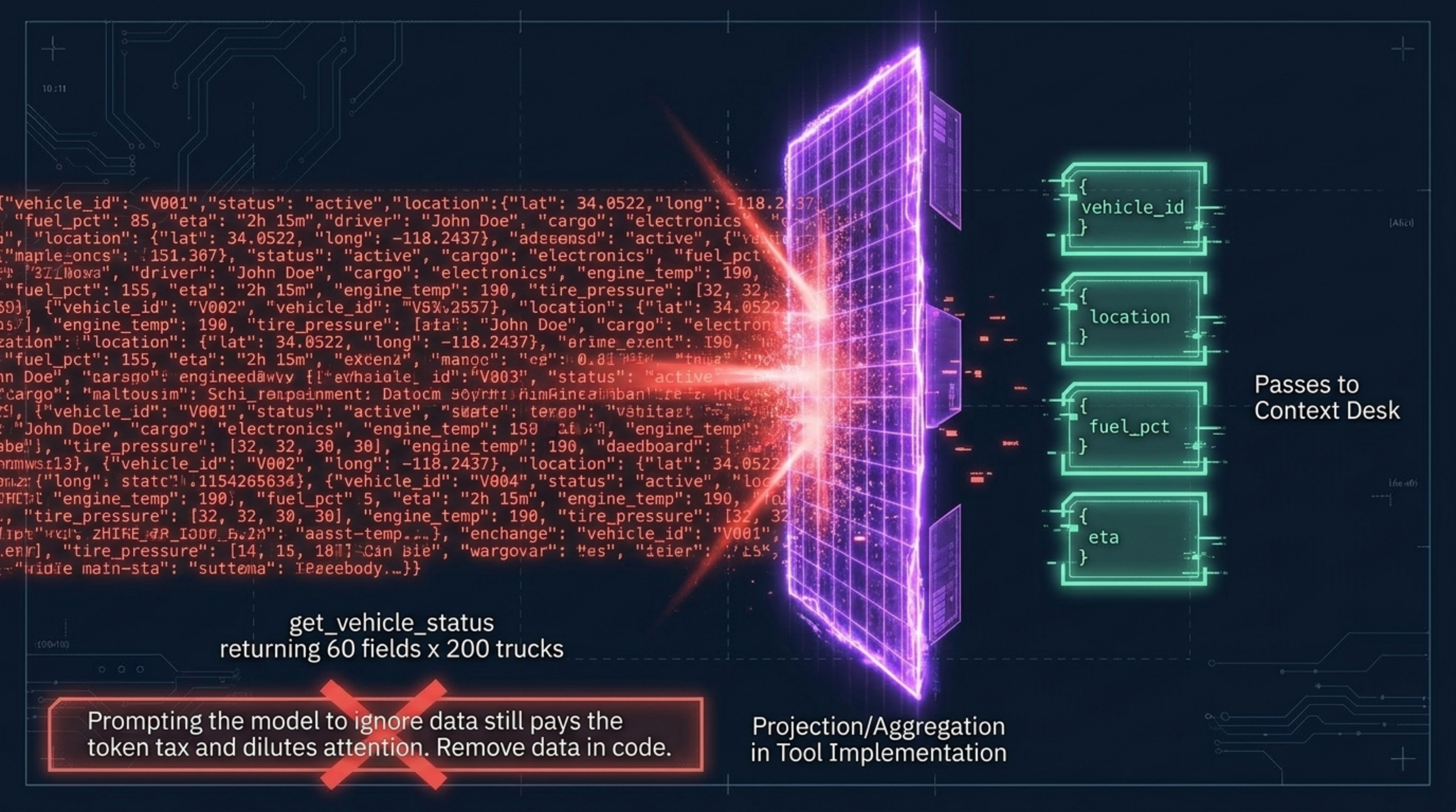
Compressed Summary

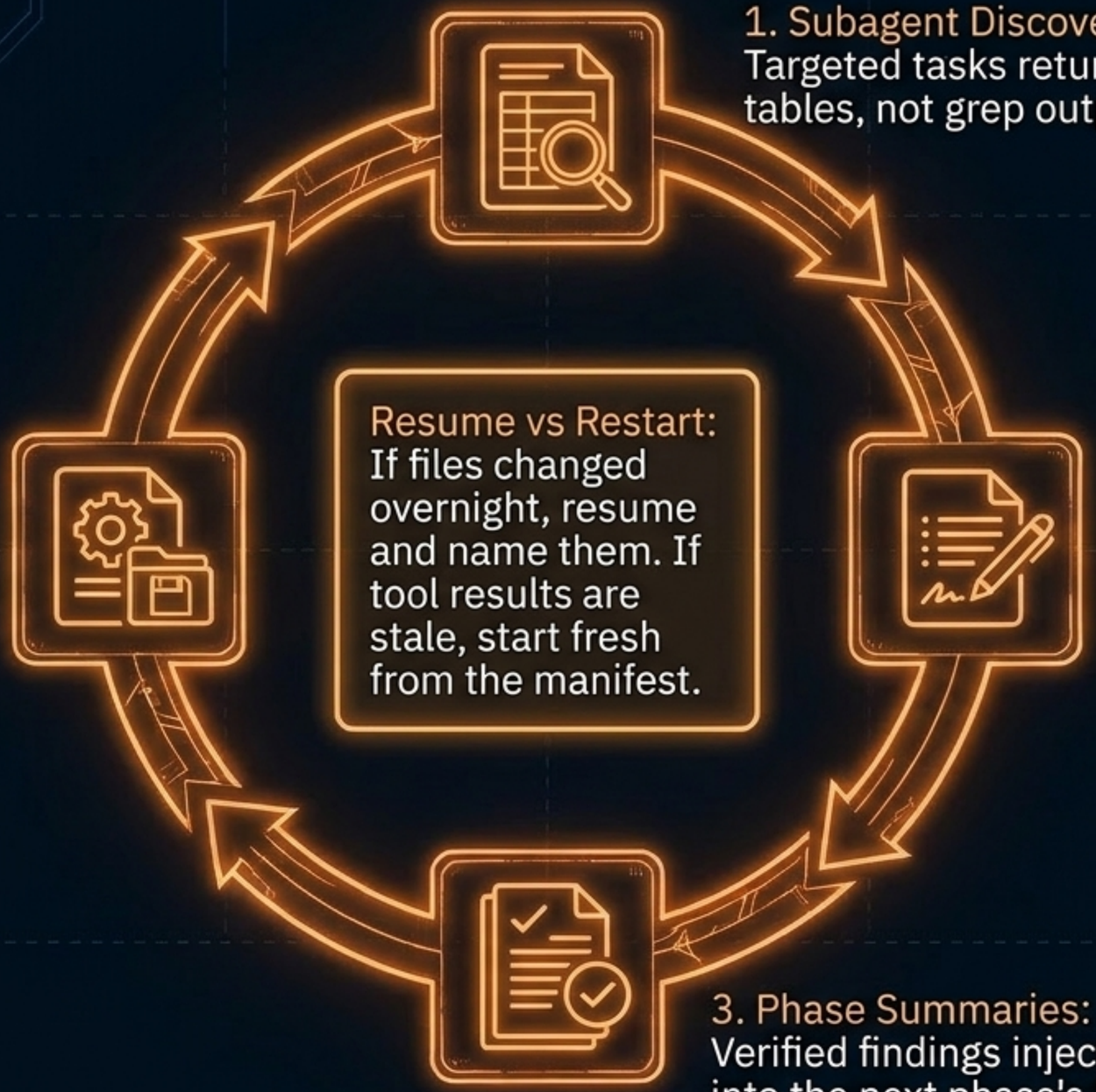
Summarize old turns. Lossy by design. Tells summarizer to preserve open decisions/file paths.

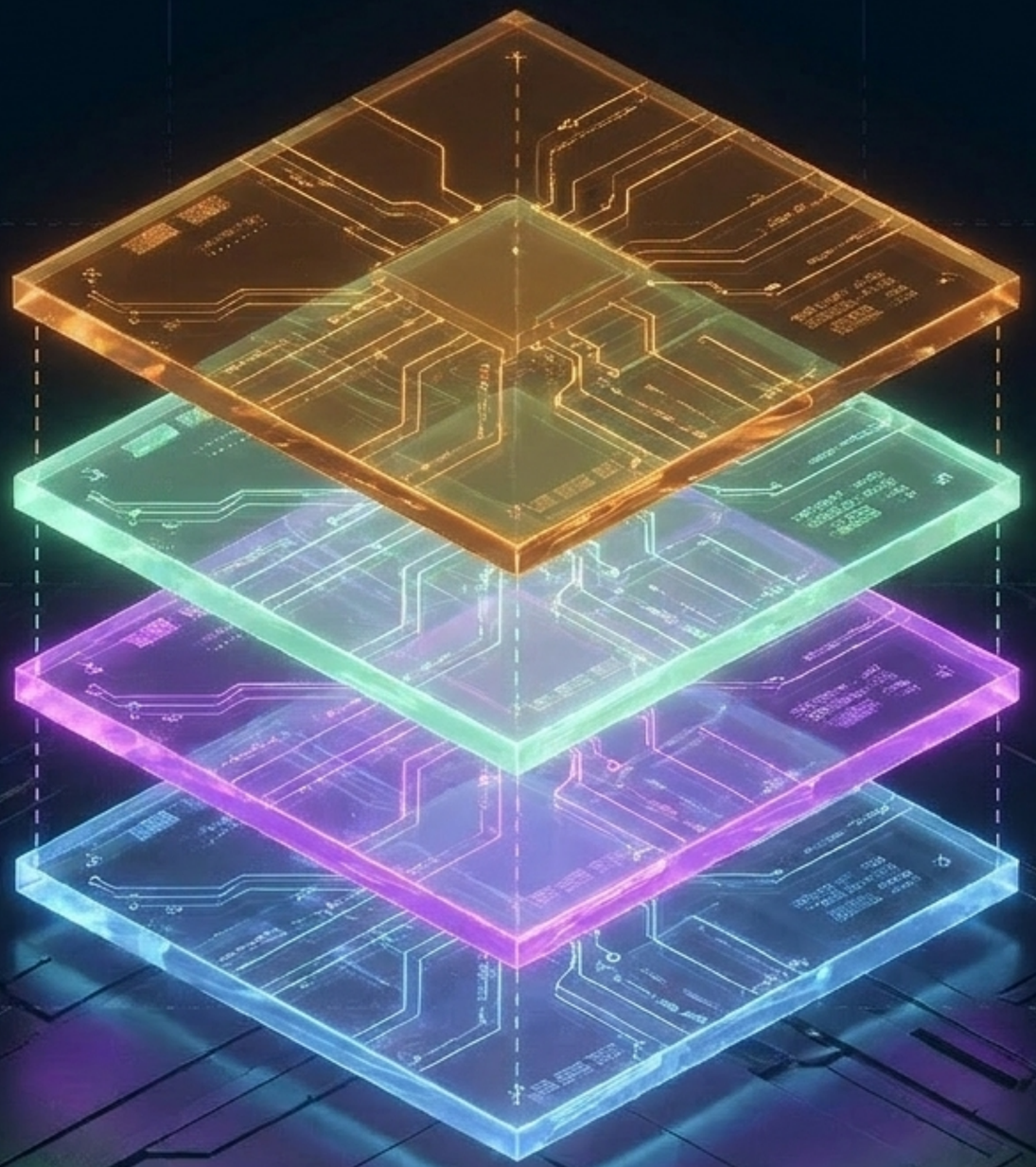
Context Editing



Clear stale tool results. Targeted removal. Tool payloads dominate and are re-fetchable. Must leave a short placeholder breadcrumb.


{
vehicle_id
}
{
location
}
{
fuel_pct
}
{
eta
}
Passes to
Context Desk
get_vehicle_status
returning 60 fields x 200 trucks
Prompting the model to ignore data still pays the
token tax and dilutes attention. Remove data in code.
Projection/Aggregation
in Tool Implementation


1. Subagent Discovery:
Targeted tasks return structured tables, not grep output.
2. Scratchpad MIGRATION_NOTES.md:
Coordinator writes decisions to file, re-reads before phases.
3. Phase Summaries:
Verified findings injected into the next phase's brief.
4. Crash-Recovery Manifest:
Writing progress to .agent-state/manifest.json.
Resume vs Restart:
If files changed overnight, resume and name them. If tool results are stale, start fresh from the manifest.

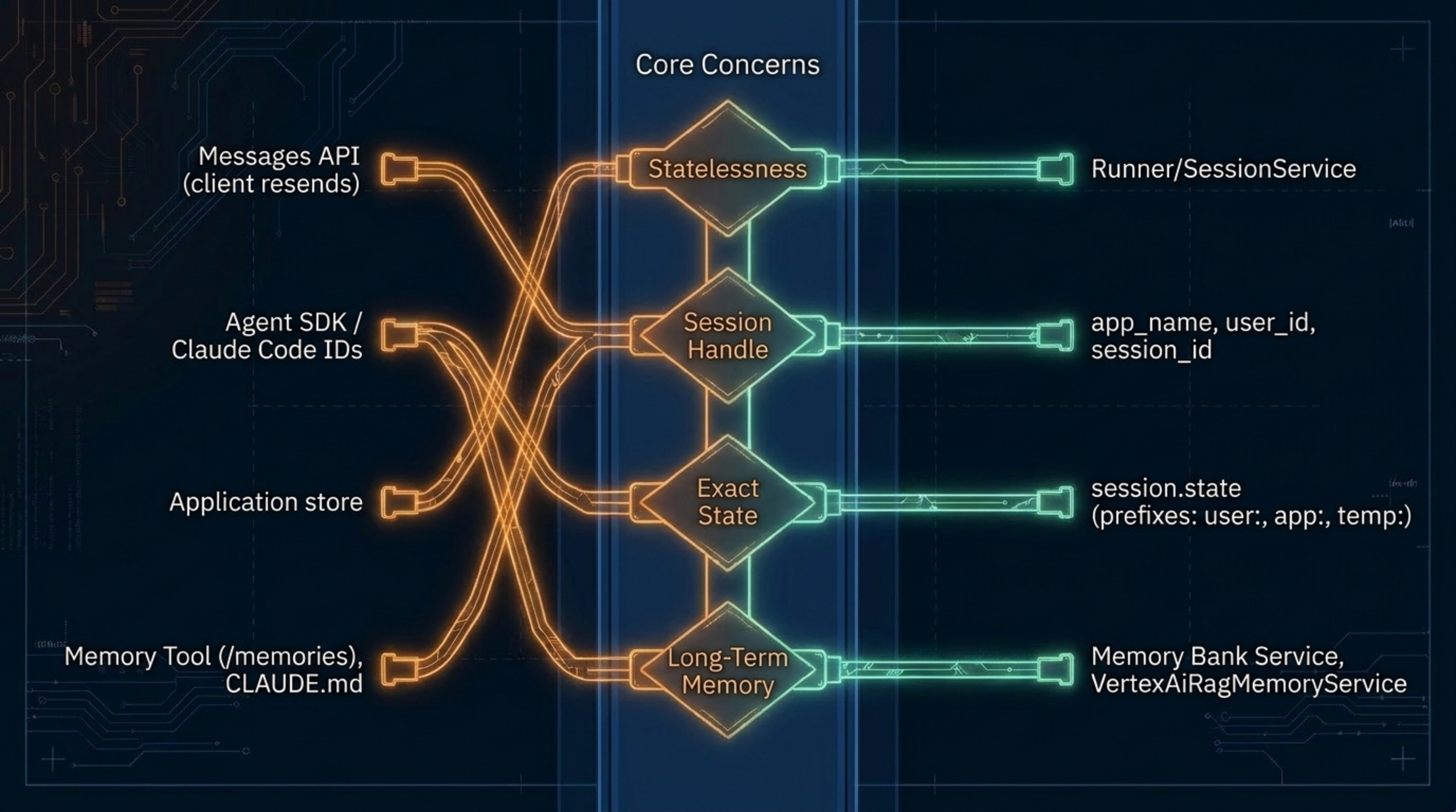


Session (History)
The ordered turns/tool calls.
Lifetime: One conversation.
Recall: Replayed/compacted.

State (Exact KV Data)
Lifetime: Session/User/App.
Recall: Deterministic read by key.
Holds tickets, approved budgets.

Memory (Extracted Facts)
Lifetime: Long-term.
Recall: Search/probabilistic.
Needs consolidation and expiry.

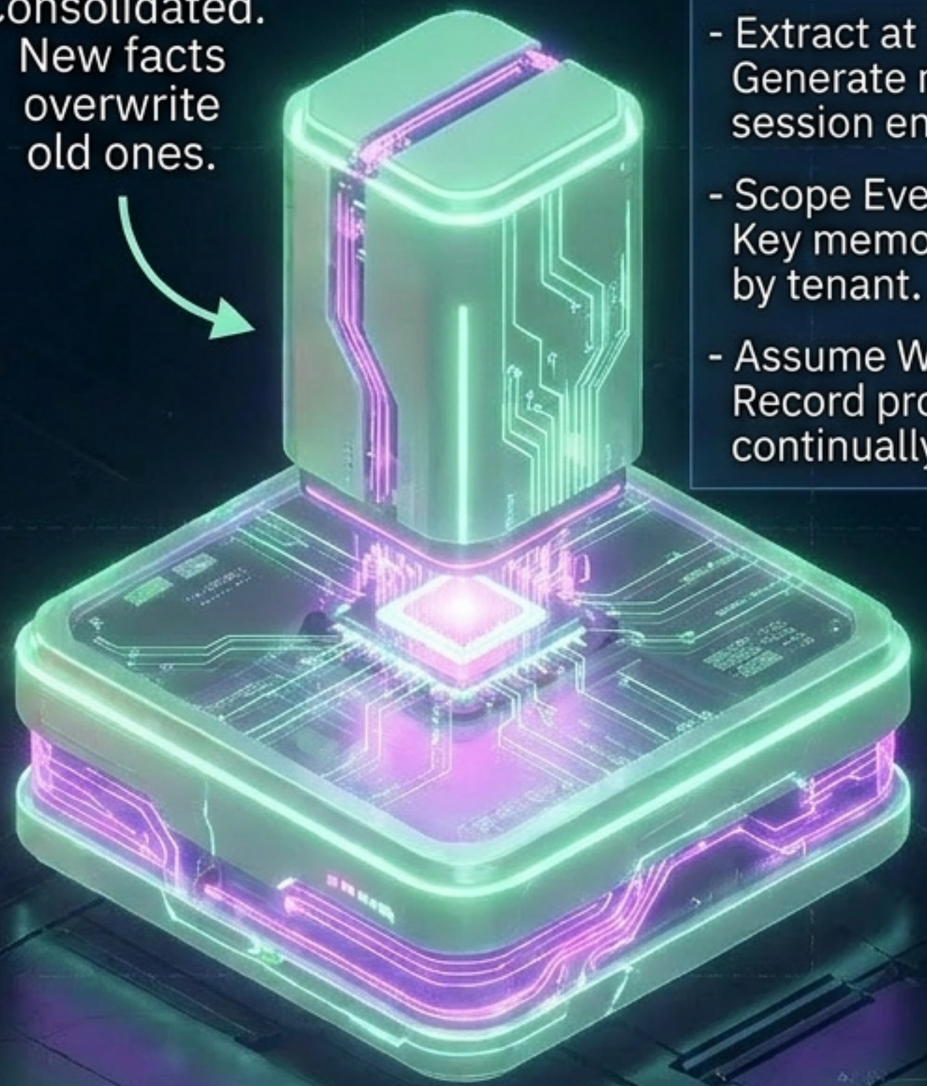
Artifacts (Large/Binary Outputs)
Scratchpads, manifests.
Lifetime: Until deleted.
Recall: Path/handle.


Core Concerns
Messages API
(client resends)
Agent SDK /
Claude Code IDs
Application store
Memory Tool (/memories),
CLAUDE.md
Statelessness
Session
Handle
Exact
State
Long-Term
Memory
Runner/SessionService
app_name, user_id,
session_id
session.state
(prefixes: user:, app:, temp:)
Memory Bank Service,
VertexAiRagMemoryService

The Anti-Pattern:
Append-Only.
Transcript dumps ruin probabilistic search.

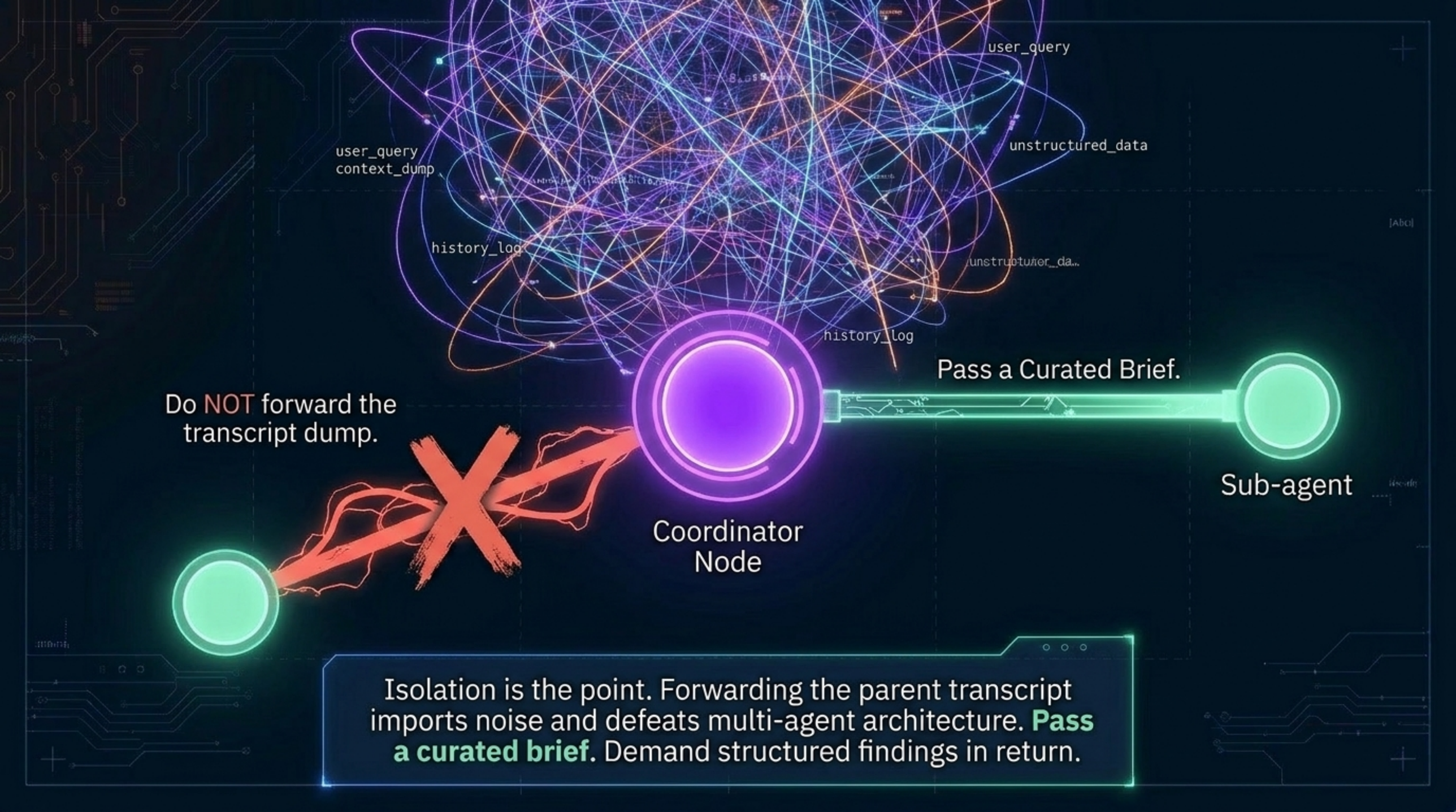


The Ideal:
Consolidated.
New facts overwrite old ones.



Memory Write Policies:

- Extract at Boundaries: Generate memories at session end.
- Scope Every Record: Key memories strictly by tenant.
- Assume Window Resets: Record progress continually.



Isolation is the point. Forwarding the parent transcript imports noise and defeats multi-agent architecture. **Pass a curated brief**. Demand structured findings in return.

Bad Handoff

Check this supplier from earlier in the chat...

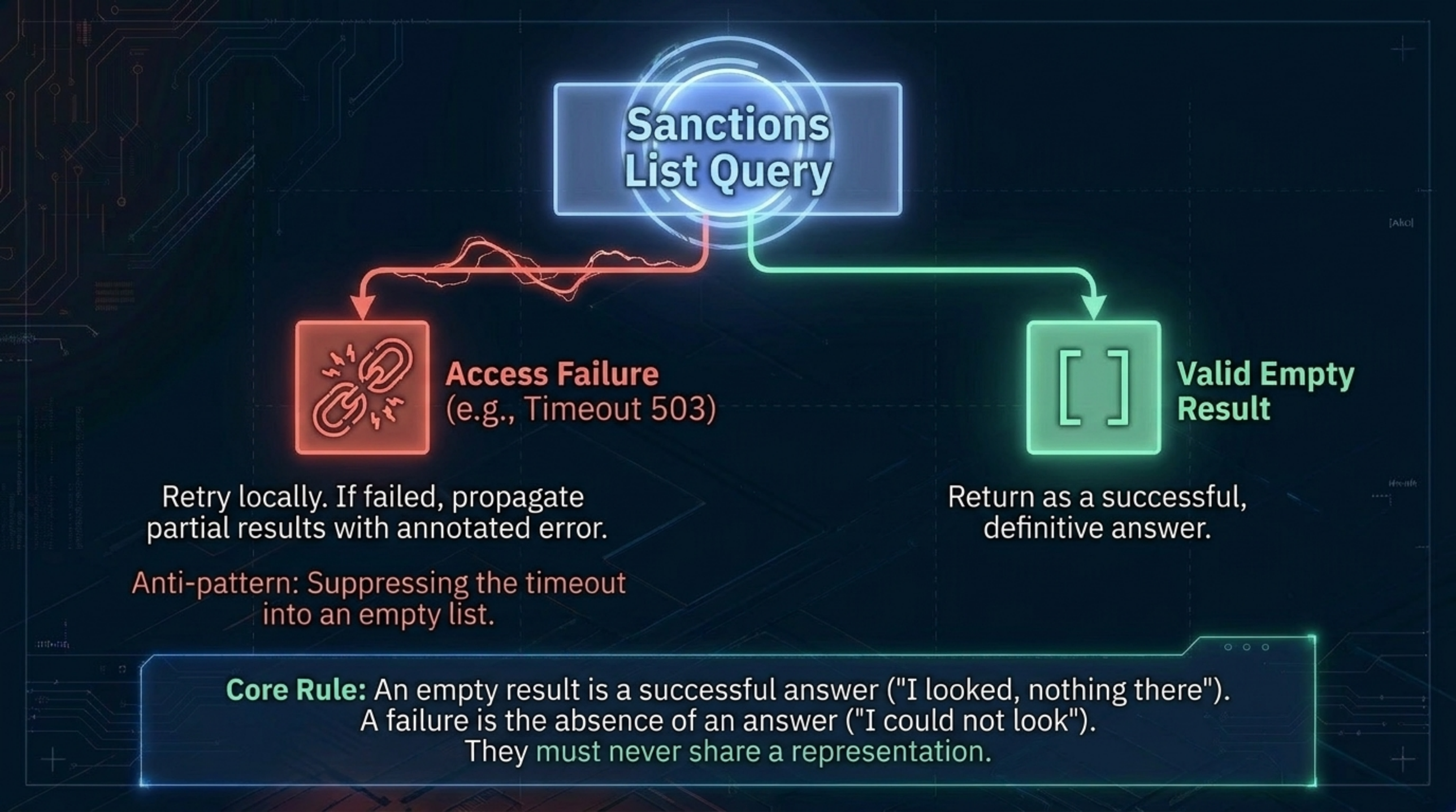
- Subagent searches the wrong company.
- Returns unstructured prose.
- Parent cannot merge or cite sources.

Good Handoff
(Kestrel Procurement)

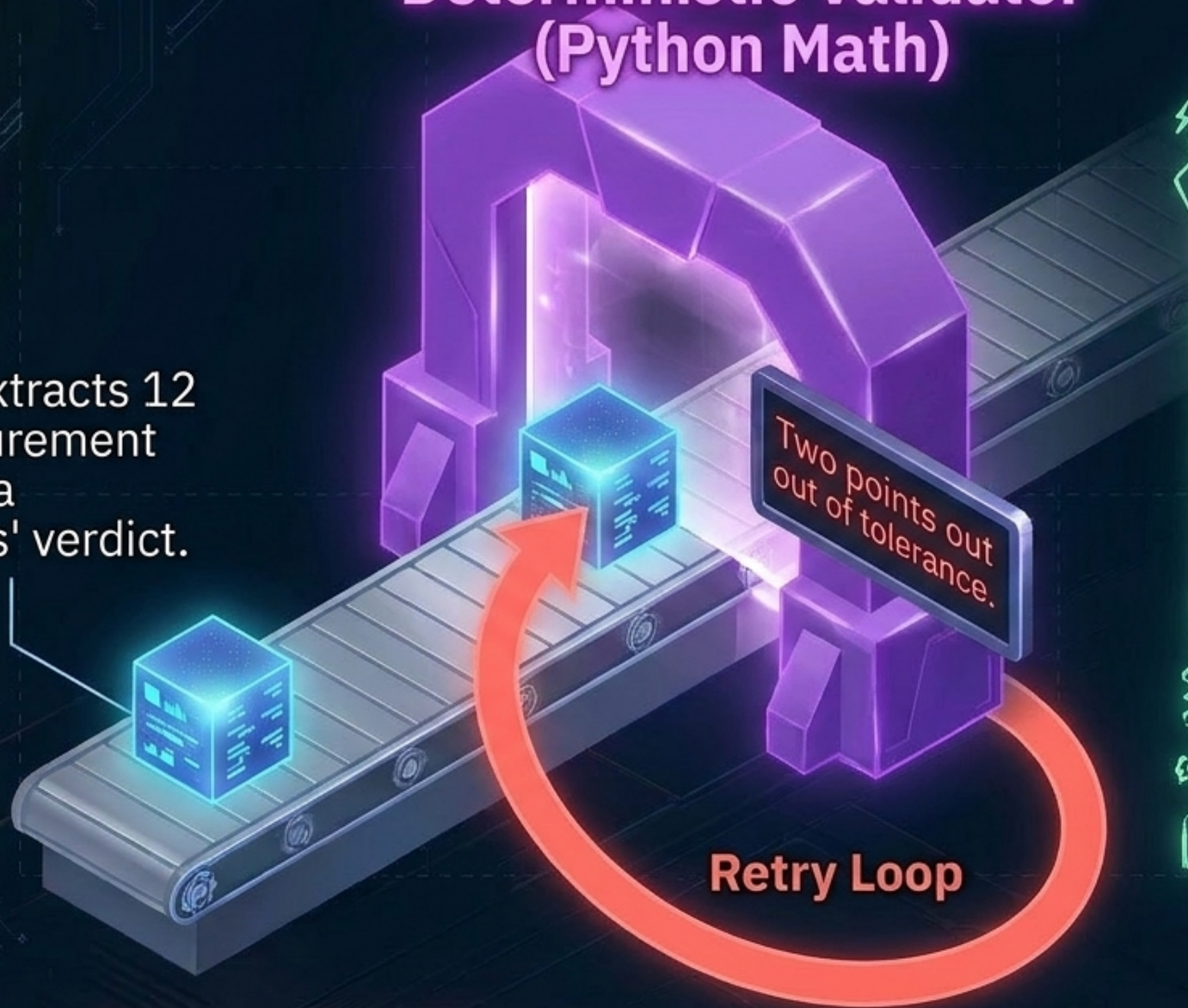
{ Coordinator sends explicit brief with exact identifiers. }

Registry | News | Sanctions

{ Subagents return structured schemas: Source, Date, Exact Claims, Coverage Gaps. }


Sanctions List Query
Access Failure
(e.g., Timeout 503)
Retry locally. If failed, propagate partial results with annotated error.
Anti-pattern: Suppressing the timeout into an empty list.
Valid Empty Result
Return as a successful, definitive answer.
Core Rule: An empty result is a successful answer ("I looked, nothing there").
A failure is the absence of an answer ("I could not look").
They must never share a representation.

Deterministic Validator (Python Math)

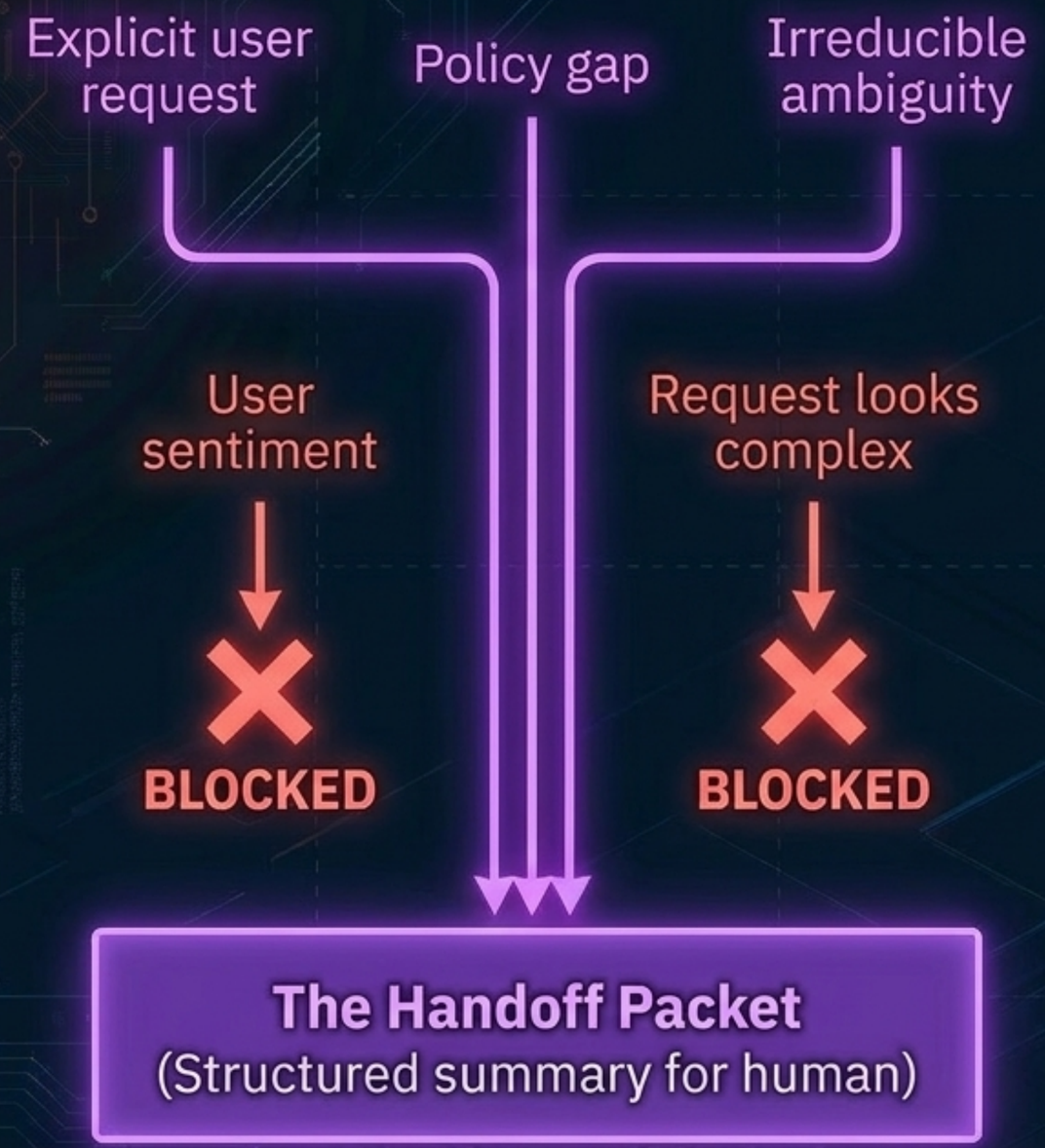


Retry Mechanics

Do: Retry with exact failure rules (Point 3 out of tolerance).

Don’t: Ask the model 'Are you sure?'

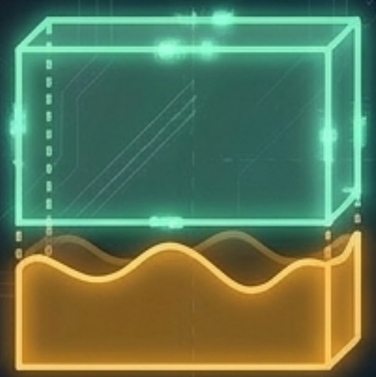
Null Handling: If genuinely missing, retry fails. Require a nullable field + 'reason': 'not_stated'.





Self-reported confidence is not calibrated.
Measure per segment via stratified sampling.
Threshold on observable criteria, not sentiment.

Caching Economics



Stable Prefix:
Tools -> System ->
Fixed Examples

Dynamic: Timestamps,
User Data

Breakpoints matter. A timestamp in the system prompt destroys cache hit rates.

Provenance



Conflicts stay visible.
Dates explain conflicts.

Security Threat Map

Memory Poisoning
(Saving malicious instructions as facts)

Indirect Prompt Injection
(Via tool payloads)

Extract facts only, validate writes, scope by tenant.